Absolutely. Below is a practical operator runbook: **what to run, in what order, what to look for, and when it is safe to continue**. It assumes the reader has never seen the underlying documentation.

VPP CN10K DPU — Ansible Execution Runbook

Project: `vpp_cn10k_ansible`
Controller: Ansible laptop
Target DPU: `xeon3-oct-112`
SSH user: `dao-test`
OS: Ubuntu 22.04
Ansible: 2.10.8
VPP: CN10K build
Data ports:

Purpose	PCI	Interface
No-carrier test port	`0002:02:00.0`	`enP2p2s0`
Live 25G port	`0002:04:00.0`	`enP2p4s0`

---

0. Prerequisites

Perform these steps on the **Ansible/controller laptop**, not on the DPU.

Go to the project

```
cd ~/Desktop/IITH/VPP/vpp_cn10k_ansible
```

Check the project:

```
tree
```

Expected structure:

```
vpp_cn10k_ansible/
├── ansible.cfg
├── inventory.ini
├── group_vars_all.yml
├── site.yml
├── README.md
└── playbooks/
    ├── 01_baseline.yml
    ├── 02_purge.yml
    ├── 03_install.yml
    ├── 04_configure_02.yml
    ├── 05_test_02.yml
    ├── 06_kernel_crosscheck.yml
    ├── 07_restore_02.yml
    ├── 08_prepare_04.yml
    ├── 09_configure_04.yml
    └── 10_verify_04.yml
```

Step 1 — Baseline Check

Playbook:

`playbooks/01_baseline.yml`

This is the first actual operation against the DPU. It is a baseline/read-only check.

1.1 Test Ansible connectivity

```
ansible dpu -m ping
```

Expected:

```
xeon3-oct-112 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

If this fails, **stop here**.

1.2 Check playbook syntax

```
ansible-playbook --syntax-check playbooks/01_baseline.yml
```

Expected:

```
playbook: playbooks/01_baseline.yml
```

1.3 Run the baseline

```
ansible-playbook playbooks/01_baseline.yml
```

The playbook checks the current state, including:

- VPP processes
- VPP/DPDK/firmware packages
- VPP sysctl configuration
- `/home/dao-test/vpp`
- Smart-NIC/secgw/nginx processes
- PCI drivers
- hugepages

Expected result:

```
failed=0
unreachable=0
```

Do **not** continue if the baseline playbook fails.

Step 2 — Purge Existing VPP Installation

Playbook:

`playbooks/02_purge.yml`

This is a **destructive step**.

Before running it, make sure the required owner/operator sign-off has been obtained.

Check the destructive-operation variable:

```
grep -n "allow_destructive_changes" group_vars_all.yml
```

For the purge operation, it must be enabled:

```
allow_destructive_changes: true
```

2.1 Run the purge

```
ansible-playbook playbooks/02_purge.yml -k -K
```

You will be asked for:

```
SSH password:
BECOME password[defaults to SSH password]:
```

Enter the appropriate `dao-test` SSH password and sudo/become password.

Expected result

The playbook should simulate and then perform the VPP purge.

The purge selection should contain:

```
The following packages will be REMOVED:
  vpp-24.02.0-cn10k*
```

The final recap should look like:

```
PLAY RECAP
xeon3-oct-112 : ok=8 changed=2 unreachable=0 failed=0
```

2.2 Verify cleanup

The playbook verifies that VPP leftovers are gone, including:

```
/etc/sysctl.d/80-vpp.conf.disabled
/run/vpp
/home/dao-test/vpp
```

Messages such as:

```
ls: cannot access '/etc/sysctl.d/80-vpp.conf.disabled'
```

are expected if the playbook is checking that the file no longer exists.

It also verifies that the Marvell packages remain installed:

```
cpt-firmware-cn10k
dpdk
dpdk-23.11-cn10k
oct-ep-target-cn10k
```

The final output should indicate that the VPP binaries are gone:

```
vpp binaries gone
```

If the purge fails, **stop here**.

Step 3 — Install VPP / Prepare Package Sysctl

The Ansible playbook implementing the installation stage is:

```
playbooks/03_install.yml
```

Run:

```
ansible-playbook playbooks/03_install.yml -k -K
```

Enter:

```
SSH password:
BECOME password[defaults to SSH password]:
```

What the playbook checks

It performs the documented pre-install checks:

- Marvell repository
- application processes
- no-carrier PCI driver
- VFIO group usage
- hugepage baseline

It then installs VPP.

Expected successful result

The recap should be:

```
PLAY RECAP
xeon3-oct-112 : ok=14 changed=2 unreachable=0 failed=0
```

The package verification should show:

```
Status: install ok installed
Version: 24.09.0
```

VPP binaries should exist:

```
/usr/bin/vpp
/bin/vpp
/usr/bin/vppctl
/bin/vppctl
```

The CN10K VPP plugin should exist:

```
/lib/aarch64-linux-gnu/vpp_plugins/dev_octeon_plugin.so
```

3.1 Verify package sysctl handling

The package-created sysctl file contains:

```
vm.nr_hugepages=1024
vm.max_map_count=3096
vm.hugetlb_shm_group=0
kernel.shmmax=2147483648
```

The playbook disables the package sysctl file as documented.

The live values should remain approximately:

```
vm.nr_hugepages = 24
vm.max_map_count = 65530
vm.hugetlb_shm_group = 0
kernel.shmmax = 18446744073692774399
```

Do not manually change these values.

Step 4 — Create VPP Configuration for `0002:02:00.0`

Playbook:

```
playbooks/04_configure_02.yml
```

Run:

```
ansible-playbook playbooks/04_configure_02.yml -k -K
```

This creates the VPP configuration used for the initial **no-carrier** test port:

```
0002:02:00.0
```

Configuration file:

```
/home/dao-test/vpp/startup-crb106-512m.conf
```

Expected result

The playbook should complete with:

```
failed=0
unreachable=0
```

The configuration should contain the documented configuration lines.

The verification should report:

```
configuration lines: 40
```

and:

```
no page options (expected)
```

Hugepages should be available, with the previously verified state:

```
2048kB 133
524288kB 23
```

Step 5 — Start and Verify VPP on the No-Carrier Port

Playbook:

```
playbooks/05_test_02.yml
```

Run:

```
ansible-playbook playbooks/05_test_02.yml -k -K
```

This starts VPP using:

```
/home/dao-test/vpp/startup-crb106-512m.conf
```

5.1 VPP process verification

Expected VPP process:

```
vpp -c /home/dao-test/vpp/startup-crb106-512m.conf
```

VPP version observed during testing:

```
vpp v24.02.0-86~g7a2e88c83
```

5.2 Plugin verification

Expected plugins include:

```
ping_plugin.so
dev_octeon_plugin.so
```

5.3 Hardware verification

The VPP hardware interface should show:

```
eth0
```

with:

```
Driver: octeon
PCI: 0002:02:00.0
```

The port is expected to be **down/no-carrier**.

This is intentional at this stage.

Expected characteristics:

```
Link speed: unknown
Link: down
```

5.4 Hugepage verification

After VPP starts, the observed state was:

```
2048kB 133
524288kB 21
```

5.5 SR-IOV verification

The observed state was:

```
totalvfs 3
numvfs 0
```

The playbook completed successfully:

```
ok=27 changed=2 unreachable=0 failed=0 skipped=0
```

If this stage succeeds, proceed to the kernel cross-check.

Step 6 — Kernel Cross-Check

Playbook:

`playbooks/06_kernel_crosscheck.yml`

Run:

```
ansible-playbook playbooks/06_kernel_crosscheck.yml -k -K
```

This temporarily stops VPP and checks the same PCI devices from the Linux/kernel side.

6.1 No-carrier device

PCI:

`0002:02:00.0`

Expected driver:

`rvu_nicpf`

The corresponding interface is:

`enP2p2s0`

Expected state:

```
state DOWN
carrier 0
```

6.2 No-carrier ethtool check

```
Speed: Unknown
Duplex: Half
Link detected: no
```

This confirms that this port has no physical carrier.

6.3 Live port check

PCI:

`0002:04:00.0`

Interface:

`enP2p4s0`

Expected:

```
Speed: 25000Mb/s
Duplex: Full
Link detected: yes
```

The kernel log should also show:

```
enP2p4s0 NIC Link is UP 25000 Mbps Full duplex
```

Successful result

The verified run completed:

```
ok=24 changed=2 unreachable=0 failed=0 skipped=3
```

Step 7 — Restore the No-Carrier Device to VFIO

Playbook:

```
playbooks/07_restore_02.yml
```

Run:

```
ansible-playbook playbooks/07_restore_02.yml -k -K
```

This returns:

```
0002:02:00.0
```

to:

```
vfio-pci
```

Verify

Expected:

```
0002:02:00.0 driver -> vfio-pci
```

The management interface must remain functional.

Expected default route:

```
default via 10.68.0.1 dev eth5
```

The live data port should still be:

```
enP2p4s0
10.65.0.190/24
UP
```

Nginx should still be running at this point.

Successful result:

```
ok=9 changed=4 unreachable=0 failed=0 skipped=0
```

Step 8 — Save State and Stop Nginx

Playbook:

```
playbooks/08_prepare_04.yml
```

This is another **destructive operation**.

Make sure:

```
allow_destructive_changes: true
```

is set before proceeding.

Run:

```
ansible-playbook playbooks/08_prepare_04.yml -k -K
```

What gets saved

The playbook creates:

```
/home/dao-test/vpp/restore/
```

and saves state including:

```
nginx.cmd
nginx.cwd
pci-04.state
```

It records the current live-port state before stopping nginx.

Nginx stop

The playbook stops:

```
nginx-tls.service
```

After the stop, expected:

```
nginx gone
```

and:

```
dtap gone
```

The playbook also checks:

- VFIO device usage
- Smart-NIC/secgw/nginx processes
- hugepage state

Important observed state

Before stopping nginx:

`sriov_numvfs = 0`

The live interface was:

`enP2p4s0`

with:

`10.65.0.190/24`

The playbook completed:

`ok=11 changed=4 unreachable=0 failed=0 skipped=0`

A ping check performed during preparation showed 100% loss to `10.65.0.192`; that check was explicitly non-fatal in the playbook, so it did **not** constitute a failure.

Step 9 — Configure the Live 25G Port

Playbook:

`playbooks/09_configure_04.yml`

Run:

```
ansible-playbook playbooks/09_configure_04.yml -k -K
```

This is the point where the actual live 25G port is handed to VPP.

Live PCI device:

`0002:04:00.0`

9.1 Verify PCI binding

Expected:

`0002:04:00.0 driver -> vfio-pci`

IOMMU group:

```
10
```

9.2 Verify Smart-NIC configuration

The verified command/configuration uses:

```
dao-smart-nic -c 0xf -a 0002:03:00.0 -a 0002:01:00.1
```

9.3 Verify management network

Management interface:

```
eth5
```

Expected:

```
eth5 UP
10.68.0.190/22
```

The default route should remain through:

```
10.68.0.1
```

9.4 Verify live VPP configuration

The live-port configuration is:

```
/home/dao-test/vpp/startup-crb106-p04.conf
```

It should contain:

```
20: dev pci/0002:04:00.0
```

Successful result:

```
ok=9 changed=4 unreachable=0 failed=0 skipped=0
```

Step 10 — Start VPP on the Live 25G Port and Verify Traffic

Playbook:

```
playbooks/10_verify_04.yml
```

Run:

```
ansible-playbook playbooks/10_verify_04.yml -k -K
```

This playbook starts VPP using:

```
/home/dao-test/vpp/startup-crb106-p04.conf
```

Step 11 — Verify the Live VPP Hardware

Expected VPP hardware:

```
eth0
```

Expected:

```
Link up
Link speed 25 Gbps
Driver octeon
PCI 0002:04:00.0
```

The assigned process node should be:

```
pci/0002:04:00.0-process
```

Step 12 — Verify the VPP Data IP

The verified VPP data address is:

```
10.65.0.192/24
```

The test peer is:

```
10.65.0.36
```

Ping test

The verification playbook performs the connectivity test.

The successful run produced:

```
4 packets transmitted
3 packets received
25% packet loss
```

Replies were received for:

```
icmp_seq 2
icmp_seq 3
icmp_seq 4
```

This confirms traffic is passing through the live VPP port.

Step 13 — Verify VPP Interface Counters

The live VPP interface was:

```
eth0
```

Observed:

```
rx packets 4
rx bytes   390

tx packets 4
tx bytes   372

ip4        3
```

The interface was administratively and operationally up.

Step 14 — Verify VPP Runtime and Errors

The runtime nodes observed included:

```
eth0-output     active
eth0-tx         active
eth0-rx         polling
ip4-input-no-checksum active
```

The VPP error output contained informational entries such as:

```
ip4-glean
arp-reply
```

No fatal VPP error was reported.

Final Verification

The final Ansible command was:

```
ansible-playbook playbooks/10_verify_04.yml -k -K
```

Successful final recap:

```
ok=14 changed=3 unreachable=0 failed=0 skipped=0
```

The final important state is:

```
Management:
    eth5
```

```
    10.68.0.190/22
    default route via 10.68.0.1

Live DPU port:
    PCI 0002:04:00.0
    VPP driver: octeon
    Speed: 25 Gbps
    Link: UP

VPP:
    Config: /home/dao-test/vpp/startup-crb106-p04.conf
    Data IP: 10.65.0.192/24

Connectivity:
    Peer: 10.65.0.36
    4 transmitted
    3 received
```

Complete Command Checklist

For someone running this from scratch, the Ansible execution order is:

```
cd ~/Desktop/IITH/VPP/vpp_cn10k_ansible
```

Connectivity

```
ansible dpu -m ping
```

Syntax checks

```
ansible-playbook --syntax-check playbooks/01_baseline.yml
ansible-playbook --syntax-check playbooks/02_purge.yml
ansible-playbook --syntax-check playbooks/03_install.yml
ansible-playbook --syntax-check playbooks/04_configure_02.yml
ansible-playbook --syntax-check playbooks/05_test_02.yml
ansible-playbook --syntax-check playbooks/06_kernel_crosscheck.yml
ansible-playbook --syntax-check playbooks/07_restore_02.yml
ansible-playbook --syntax-check playbooks/08_prepare_04.yml
ansible-playbook --syntax-check playbooks/09_configure_04.yml
ansible-playbook --syntax-check playbooks/10_verify_04.yml
```

Execution

```
ansible-playbook playbooks/01_baseline.yml

ansible-playbook playbooks/02_purge.yml -k -K

ansible-playbook playbooks/03_install.yml -k -K

ansible-playbook playbooks/04_configure_02.yml -k -K

ansible-playbook playbooks/05_test_02.yml -k -K

ansible-playbook playbooks/06_kernel_crosscheck.yml -k -K

ansible-playbook playbooks/07_restore_02.yml -k -K
```

```
ansible-playbook playbooks/08_prepare_04.yml -k -K

ansible-playbook playbooks/09_configure_04.yml -k -K

ansible-playbook playbooks/10_verify_04.yml -k -K
```

Important Rules for Operators

1. **Run the playbooks in order.**
2. Do not skip a failed playbook and continue.
3. Do not run `site.yml` during step-by-step execution.
4. `-k` asks for the SSH password.
5. `-K` asks for the sudo/become password.
6. Steps involving purge, device binding, or stopping services are destructive. Make sure `allow_destructive_changes` is intentionally enabled before those stages.
7. Do not manually modify VPP configuration between the Ansible stages unless the runbook explicitly instructs you to.
8. The `Ubuntu 22.04 ... /usr/bin/python` deprecation warning from Ansible 2.10.8 is **not a failure**.
9. A successful run is indicated by:

   ```
   failed=0
   unreachable=0
   ```

10. The final success condition is **the live `0002:04:00.0` port operating at 25 Gbps under VPP and passing traffic to `10.65.0.36`**.

One-line operator summary

```
CONNECT → BASELINE → PURGE → INSTALL → CONFIGURE NO-CARRIER → TEST → KERNEL
CHECK → RESTORE → STOP NGINX → CONFIGURE LIVE 25G → START VPP → VERIFY TRAFFIC
```

Do not use this runbook to skip the underlying safety/sign-off requirements; it is intended to reproduce the tested Ansible sequence exactly.